Assignment 2

1. Giới thiệu và quy định nộp bài

Ba bài tập rèn luyện kỹ năng thao tác mảng/list (`std::vector`), xâu ký tự (`std::string`); chạy trực tiếp trên máy tính, không cần bo mạch hoặc thư viện bên ngoài.

Đi kèm tài liệu mô tả này là tệp header **`bai_tap.h`** chứa khai báo nguyên mẫu (signature) của 3 hàm cần hoàn thiện:

```
#ifndef C_FOR_EMBED_PE_ASSIGNMENT2_BAI_TAP_H
#define C_FOR_EMBED_PE_ASSIGNMENT2_BAI_TAP_H

#include <vector>
#include <string>

// Bài 1: Tìm cặp phần tử có tổng bằng mục tiêu (Two Sum)
std::vector<int> two_sum(const std::vector<int>& nums, int target);

// Bài 2: Tìm tiền tố chung dài nhất (Longest Common Prefix)
std::string longest_common_prefix(const std::vector<std::string>& strs);

// Bài 3: Cộng một đơn vị vào số nguyên lớn (Plus One)
std::vector<int> plus_one(const std::vector<int>& digits);

#endif /* C_FOR_EMBED_PE_ASSIGNMENT2_BAI_TAP_H */
```

Quy cách làm bài và nộp bài:

1. Sinh viên giữ nguyên tệp `bai_tap.h` và định nghĩa cả 3 hàm đã khai báo (`two_sum`, `longest_common_prefix`, `plus_one`) vào **duy nhất 1 tệp mã nguồn** **`bai_tap.cpp`**.
2. Đầu tệp `bai_tap.cpp` bắt buộc phải có dòng `#include "bai_tap.h"`.
3. **Tệp cần nộp:** Chỉ nộp duy nhất tệp **`bai_tap.cpp`**.
4. Hệ thống chấm tự động sẽ gọi trực tiếp các hàm này; **không viết hàm `main`** bên trong `bai_tap.cpp` và không đọc dữ liệu từ bàn phím (`cin` / `scanf`).
5. Kiểm tra cú pháp và cảnh báo của tệp trước khi nộp:

```
c++ -std=c++20 -Wall -Wextra -Wpedantic -c bai_tap.cpp
```

2. Mô tả chi tiết các bài tập

Bài 1 — Cặp phần tử có tổng bằng mục tiêu (Two Sum) (3.0 điểm)

Bối cảnh bài toán

Trong các bài toán xử lý dữ liệu đo đạc và lọc tín hiệu, ta thường cần tìm cặp mẫu số liệu thỏa mãn một điều kiện tổng giá trị định trước. Cho một vector số nguyên `nums` và một giá trị mục tiêu `target`.

Hãy tìm hai vị trí (chỉ số) khác nhau trong vector sao cho tổng giá trị của hai phần tử tại vị trí đó đúng bằng `target`.

Nguyên mẫu hàm

```
std::vector<int> two_sum(const std::vector<int>& nums, int target);
```

- **Đầu vào:**
  - `nums`: tham chiếu hằng tới vector số nguyên (`const std::vector<int>&`).
  - `target`: giá trị mục tiêu kiểu số nguyên có dấu `int`.
  - Giả định: Mỗi bộ dữ liệu kiểm thử hợp lệ có đúng một nghiệm duy nhất; không sử dụng cùng một phần tử hai lần.
- **Đầu ra:**
  - `std::vector<int>` gồm đúng 2 phần tử `{i, j}` chứa hai chỉ số tìm được (thứ tự `{i, j}` hay `{j, i}` đều được chấp nhận).
  - Trường hợp `nums` có ít hơn 2 phần tử hoặc không tìm thấy nghiệm hợp lệ, hàm trả về vector rỗng `{}`.
- **Yêu cầu triển khai:**
  - Trả về trực tiếp đối tượng `std::vector<int>` chứa hai chỉ số.
  - Sinh viên được tự do lựa chọn và sáng tạo giải pháp thuật toán phù hợp bên trong `bai_tap.cpp`. Các giải thuật khác nhau sẽ có điểm số khác nhau. Theo đó, giải thuật càng tối ưu thì điểm càng cao.

Các ví dụ minh họa

- **Ví dụ 1 (Trường hợp cơ bản):**
  - **Đầu vào:** `nums = {2, 7, 11, 15}`, `target = 9`
  - **Đầu ra:** `{0, 1}` (hoặc `{1, 0}`)
  - **Giải thích:** `nums[0] + nums[1] = 2 + 7 = 9`. Hai chỉ số cần tìm là `0` và `1`.
- **Ví dụ 2 (Cặp số nằm ở giữa mảng):**
  - **Đầu vào:** `nums = {3, 2, 4}`, `target = 6`
  - **Đầu ra:** `{1, 2}`
  - **Giải thích:** `nums[1] + nums[2] = 2 + 4 = 6`. Lưu ý không được chọn `{0, 0}` dù `nums[0] + nums[0] = 3 + 3 = 6` vì hai chỉ số bắt buộc phải khác nhau ($i \neq j$).
- **Ví dụ 3 (Hai phần tử có cùng giá trị ở hai vị trí khác nhau):**
  - **Đầu vào:** `nums = {3, 3}`, `target = 6`
  - **Đầu ra:** `{0, 1}`
  - **Giải thích:** `nums[0] = 3` và `nums[1] = 3` nằm ở hai vị trí khác nhau, tổng bằng `6`.
- **Ví dụ 4 (Mảng chứa số âm):**
  - **Đầu vào:** `nums = {-1, -2, -3, -4, -5}`, `target = -8`
  - **Đầu ra:** `{2, 4}`
  - **Giải thích:** `nums[2] = -3` và `nums[4] = -5`, tổng là `(-3) + (-5) = -8`.

- **Ví dụ 5 (Cặp số ở hai đầu mút đối nhau qua 0):**
  - **Đầu vào:** `nums = {100, 20, 30, 40, -100}`, `target = 0`
  - **Đầu ra:** `{0, 4}`
  - **Giải thích:** `nums[0] + nums[4] = 100 + (-100) = 0`.
- **Ví dụ 6 (Trường hợp không có nghiệm hoặc mảng quá ngắn):**
  - **Đầu vào:** `nums = {1, 2, 3}`, `target = 100` (hoặc `nums = {5}`, `target = 5`)
  - **Đầu ra:** `{}` (vector rỗng)

Bài 2 — Tiền tố chung dài nhất (Longest Common Prefix) (3.5 điểm)

Bối cảnh bài toán

Trong hệ thống nhúng giao tiếp qua mạng hoặc cổng nối tiếp UART, các bộ phân tích cú pháp gói tin (parser) thường phải xác định tiền tố nhận dạng chung của một tập hợp các chuỗi mã lệnh (chẳng hạn tiền tố `"AT+CW"` trong nhóm tập lệnh WiFi). Cho vector `strs` chứa danh sách các xâu ký tự `std::string`. Hãy tìm tiền tố chung dài nhất xuất hiện ở đầu tất cả các xâu trong vector.

Nguyên mẫu hàm

```
std::string longest_common_prefix(const std::vector<std::string>& strs);
```

- **Đầu vào:**
  - `strs`: tham chiếu hằng tới vector các xâu ký tự (`const std::vector<std::string>&`).
- **Đầu ra:**
  - Đối tượng `std::string` biểu diễn tiền tố chung dài nhất.
  - Trường hợp vector `strs` rỗng hoặc giữa các xâu không có ký tự chung nào ở đầu, hàm trả về xâu rỗng `""`.
- **Yêu cầu triển khai:**
  - Xử lý đầy đủ các trường hợp: vector rỗng, vector chỉ có 1 xâu, hoặc chứa xâu rỗng `""`.
  - Tận dụng các phương thức chuẩn của `std::string` (như `.substr()`, `.size()`, `[]`).

Các ví dụ minh họa

- **Ví dụ 1 (Có tiền tố chung một phần):**
  - **Đầu vào:** `strs = {"flower", "flow", "flight"}`
  - **Đầu ra:** `"fl"`
  - **Giải thích:** Cả 3 xâu đều bắt đầu bằng `'f'` và `'l'`. Đến vị trí chỉ số 2, `"flower"` và `"flow"` có `'o'`, còn `"flight"` có `'i'`. Do đó tiền tố chung dừng lại ở `"fl"`.
- **Ví dụ 2 (Không có bất kỳ ký tự chung nào ở đầu):**
  - **Đầu vào:** `strs = {"dog", "racecar", "car"}`
  - **Đầu ra:** `""`
  - **Giải thích:** Ký tự đầu tiên của 3 xâu là `'d'`, `'r'`, `'c'` khác nhau hoàn toàn, trả về xâu rỗng `""`.

- **Ví dụ 3 (Một xâu ngắn là tiền tố của toàn bộ các xâu dài hơn):**
  - **Đầu vào:** `strs = {"thiet_bi", "thiet_bi_do", "thiet_bi_dieu_khien"}`
  - **Đầu ra:** `"thiet_bi"`
  - **Giải thích:** Toàn bộ xâu `"thiet_bi"` xuất hiện trọn vẹn ở đầu hai xâu còn lại.
- **Ví dụ 4 (Mảng chỉ có đúng 1 xâu duy nhất):**
  - **Đầu vào:** `strs = {"sensor"}`
  - **Đầu ra:** `"sensor"`
- **Ví dụ 5 (Mảng chứa xâu rỗng):**
  - **Đầu vào:** `strs = {"", "abc", "abd"}`
  - **Đầu ra:** `""`
- **Ví dụ 6 (Vector rỗng không có phần tử):**
  - **Đầu vào:** `strs = {}`
  - **Đầu ra:** `""`

Bài 3 — Cộng một đơn vị vào số nguyên lớn (Plus One) (3.5 điểm)

Bối cảnh bài toán

Trong các bộ đo đếm chu kỳ hoạt động hoặc đồng hồ ghi nhận năng lượng của thiết bị nhúng (energy meter), số liệu tích lũy thường vượt quá giới hạn biểu diễn của số nguyên phần cứng 32-bit hoặc 64-bit. Do đó, giá trị này được lưu trữ dưới dạng mảng các chữ số thập phân. Cho vector `digits` biểu diễn một số nguyên không âm. Chữ số có trọng số cao nhất (hàng cao nhất) nằm ở `digits[0]`, và chữ số hàng đơn vị nằm ở cuối vector (`digits.back()`). Hãy thực hiện phép toán cộng thêm 1 đơn vị vào số nguyên này và trả về vector các chữ số kết quả.

Nguyên mẫu hàm

```
std::vector<int> plus_one(const std::vector<int>& digits);
```

- **Đầu vào:**
  - `digits`: tham chiếu hằng tới vector các chữ số (`const std::vector<int>&`), mỗi phần tử trong khoảng từ 0 đến 9.
  - Dãy chữ số không chứa số 0 ở đầu, ngoại trừ trường hợp chính số 0 (`{0}`).
- **Đầu ra:**
  - `std::vector<int>` chứa các chữ số của kết quả sau khi cộng 1.
  - Nếu `digits` rỗng, hàm trả về `{}`.
- **Yêu cầu triển khai:**
  - Duyệt và tính toán phép cộng từ phải sang trái (từ hàng đơn vị sang hàng cao nhất), quản lý biến nhớ (carry).
  - Xử lý trường hợp đặc biệt khi toàn bộ các chữ số ban đầu đều là 9 (ví dụ: `999` + 1 = `1000`): kích thước vector kết quả tăng thêm 1 phần tử so với ban đầu. Không sử dụng lệnh `goto`.

Các ví dụ minh họa

- **Ví dụ 1 (Cộng không có nhớ ở hàng đơn vị):**
  - **Đầu vào:** `digits = {1, 2, 3}`
  - **Đầu ra:** `{1, 2, 4}`
  - **Giải thích:** Biểu diễn số 123. Khi cộng 1: 123 + 1 = 124.
- **Ví dụ 2 (Cộng có nhớ ở hàng đơn vị):**
  - **Đầu vào:** `digits = {1, 2, 9}`
  - **Đầu ra:** `{1, 3, 0}`
  - **Giải thích:** Biểu diễn số 129. Khi cộng 1: 129 + 1 = 130 (hàng đơn vị 9 + 1 = 10 viết 0 nhớ 1 sang hàng chục 2 + 1 = 3).
- **Ví dụ 3 (Nhớ lan truyền qua nhiều hàng liên tiếp nhưng không tăng số chữ số):**
  - **Đầu vào:** `digits = {1, 9, 9}`
  - **Đầu ra:** `{2, 0, 0}`
  - **Giải thích:** 199 + 1 = 200. Kích thước vẫn giữ nguyên là 3 chữ số.
- **Ví dụ 4 (Toàn bộ chữ số đều là 9 — Tăng số lượng chữ số):**
  - **Đầu vào:** `digits = {9, 9, 9}`
  - **Đầu ra:** `{1, 0, 0, 0}`
  - **Giải thích:** 999 + 1 = 1000. Cả 3 chữ số 9 chuyển thành 0 và phát sinh thêm chữ số 1 ở đầu vector. Kích thước tăng từ 3 lên 4 phần tử.
- **Ví dụ 5 (Mảng có 1 phần tử là số 9):**
  - **Đầu vào:** `digits = {9}`
  - **Đầu ra:** `{1, 0}`
- **Ví dụ 6 (Mảng có 1 phần tử là số 0):**
  - **Đầu vào:** `digits = {0}`
  - **Đầu ra:** `{1}`

---

3. Hướng dẫn tự viết hàm `main` để kiểm thử trên máy cá nhân

Các em có thể tạo một tệp riêng (ví dụ `my_test.cpp` — **không nộp tệp này**) để chạy thử nghiệm 3 hàm của mình như sau:

```cpp
#include <iostream>
#include "bai_tap.h"

int main() {
    // 1. Thử nghiệm Bài 1: two_sum
    std::vector<int> nums = {2, 7, 11, 15};
    std::vector<int> idx = two_sum(nums, 9);
    if (idx.size() == 2) {
        std::cout << "Bai 1: [" << idx[0] << ", " << idx[1] << "]\n"; // Kỳ vọng: [0, 1]
    }

    // 2. Thử nghiệm Bài 2: longest_common_prefix
    std::vector<std::string> strs = {"flower", "flow", "flight"};
    std::cout << "Bai 2: \"" << longest_common_prefix(strs) << "\"\n"; // Kỳ vọng: "fl"

    // 3. Thử nghiệm Bài 3: plus_one
    std::vector<int> digits = {9, 9, 9};
    std::vector<int> plus_res = plus_one(digits);
    std::cout << "Bai 3: ";
    for (int d : plus_res) {
        std::cout << d << " ";
    }
    std::cout << "\n"; // Kỳ vọng: 1 0 0 0

    return 0;
}
```

Biên dịch và chạy thử tất cả cùng lúc:

```
c++ -std=c++20 -Wall -Wextra -Wpedantic my_test.cpp bai_tap.cpp -o my_test
./my_test
```

---

4. Tiêu chí đánh giá

Nội dung	Điểm
Bài 1: Các ca kiểm thử đều trả về đúng kết quả	2.0
Bài 1: Sử dụng đúng kiểu dữ liệu STL `std::vector`, nắm vững tư duy tối ưu thuật toán	1.0
Bài 2: Các ca kiểm thử đều trả về đúng kết quả	2.5
Bài 2: Thao tác xâu ký tự an toàn với `std::string`, phong cách mã nguồn rõ ràng	1.0
Bài 3: Các ca kiểm thử đều trả về đúng kết quả (bao gồm trường hợp tăng số chữ số)	2.5
Bài 3: Xử lý biến nhớ chính xác, phong cách mã nguồn rõ ràng	1.0

File ca kiểm thử dùng để tính điểm không được công bố.